

Day 20 — Week 3 complete study guide

MD Mutasim Billah · 52-week Data Science to ML/AI roadmap · Week 3 review — definitions, real-world examples, implementations, and an interview-style quiz for Days 15-19

No new code was introduced this week beyond what Days 15-19 already covered. This guide is a single reference: for each day, a plain-language definition, a real-world analogy, and a runnable implementation snippet, followed by a technical and theoretical quiz written the way these ideas tend to get asked about in a data science interview.

Day 15 — Logistic regression: the first real classifier

Definition

Definition — Logistic Regression: a supervised classification algorithm that learns one weighted combination of input features and passes it through the sigmoid function to output a probability between 0 and 1, rather than an unbounded number. That probability is then compared against a threshold (0.5 by default) to produce a class label.

Real-world example

Real-world example: A bank deciding whether to approve a loan doesn't just want yes/no — it wants a risk score. Logistic regression is exactly that kind of model: it might learn that higher income pushes the approval probability up and a poor credit history pushes it down, and the bank can set its own approval cutoff (maybe 0.5, maybe stricter) depending on how much risk it's willing to accept — the same idea as choosing a decision threshold in Day 18.

Implementation

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

probs = model.predict_proba(X_test)[:, 1]    # P(class = 1) for each row
preds = model.predict(X_test)                 # probs > 0.5, by default

coefficients = pd.Series(model.coef_[0], index=X_train.columns).sort_values()
print(coefficients)
```

- `max_iter=1000` — raises the optimizer's iteration cap so it has room to converge on datasets where the default (100) isn't enough

- `model.coef_[0]` — one coefficient per feature; a negative coefficient pushes the probability toward class 0, a positive one pushes toward class 1
- logistic regression assumes a roughly linear relationship between features and the log-odds of the outcome — it won't automatically discover an interaction like "feature A matters only when feature B is high" the way a tree can

Day 16 — Decision trees and random forests

Definition

Definition — Decision Tree: a model that splits data with a sequence of yes/no questions on feature values, choosing at each step the question that best separates the classes (measured by Gini impurity), until it reaches a leaf that predicts a class.

Definition — Random Forest: an ensemble of many decision trees, each trained on a random subset of rows and features, whose predictions are combined by majority vote. Averaging cancels out much of each individual tree's overfitting.

Definition — Overfitting (train/test gap): when a model performs much better on the data it was trained on than on new, unseen data — a sign it has memorized noise and specifics of the training set rather than learning patterns that generalize.

Real-world example

Real-world example: A doctor diagnosing a patient asks a sequence of questions — "is fever present?", "is it above 102°F?", "any rash?" — each answer narrowing down the diagnosis. That's a decision tree. A random forest is the same idea but asking several different doctors (each trained slightly differently, each seeing a different subset of the patient's history) and going with the majority diagnosis — more reliable than trusting any single doctor's chain of questions.

Implementation

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

tree = DecisionTreeClassifier(max_depth=4, min_samples_leaf=10, random_state=42)
tree.fit(X_train, y_train)

train_acc = accuracy_score(y_train, tree.predict(X_train))
test_acc = accuracy_score(y_test, tree.predict(X_test))
print(f"Gap: {train_acc - test_acc:.3f}") # large gap = likely overfitting

forest = RandomForestClassifier(n_estimators=200, max_depth=4, random_state=42)
forest.fit(X_train, y_train)

importances = pd.Series(
    forest.feature_importances_, index=X_train.columns
).sort_values(ascending=False)

```

- `max_depth` caps how many questions deep a tree can go; `min_samples_leaf` requires each leaf to cover a minimum number of rows — both limit overfitting by keeping the tree from growing complex enough to memorize the training set
- `n_estimators=200` — number of trees in the forest; more trees generally stabilizes the averaged prediction, at the cost of training time
- `feature_importances_` — unlike logistic regression's signed coefficients, this is an unsigned ranking of how much each feature reduced impurity across all trees

Day 17 — Cross-validation and GridSearchCV

Definition

Definition — K-Fold Cross-Validation: splits the training data into K equal folds, trains on K-1 of them and tests on the one left out, repeating K times so every row is tested exactly once. Produces K scores instead of one, summarized as a mean and a standard deviation.

Definition — GridSearchCV: exhaustively tries every combination of hyperparameters in a given grid, scoring each combination with cross-validation, and returns whichever combination scored best on average across the folds.

Real-world example

Real-world example: Judging a restaurant off one visit is risky — maybe the chef was having an off night, or it was unusually busy. Cross-validation is like reviewing the restaurant five separate times on different nights and averaging the ratings, which is a far more trustworthy verdict than any single visit. GridSearchCV is the same idea applied to menu testing: trying every combination of recipe tweaks and judging each one across several nights before picking a winner, instead of guessing which tweak sounds best.

Implementation

```
from sklearn.model_selection import cross_val_score, GridSearchCV

scores = cross_val_score(model, X_train, y_train, cv=5)
print(f"CV mean = {scores.mean():.3f}, std = {scores.std():.3f}")

param_grid = {
    "n_estimators": [100, 200],
    "max_depth": [4, 8, None],
    "min_samples_leaf": [1, 5],
}
search = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5, n_jobs=-1)
search.fit(X_train, y_train)

print(search.best_params_, search.best_score_)
```

- `cv=5` — 5-fold cross-validation, the most common default; higher K means more folds and a more stable estimate, at the cost of more training runs
- reporting only the mean and hiding the standard deviation is a common interview trap — two models with the same mean CV score are not equally trustworthy if one has a much higher spread across folds
- `n_jobs=-1` — runs combinations in parallel across all available CPU cores instead of testing them one at a time
- the held-out test set (`X_test/y_test`) should never be touched during this process — every decision here uses `X_train/y_train` only

Day 18 — ROC, AUC, and decision thresholds

Definition

Definition — ROC Curve: a plot of true positive rate against false positive rate as the decision threshold is swept from 1.0 down to 0.0, showing how a model trades off catching real positives against incorrectly flagging negatives.

Definition — AUC (Area Under the Curve): a single number between 0 and 1 summarizing the ROC curve: the probability that a randomly chosen positive example is ranked higher than a randomly chosen negative one. 0.5 means no better than random guessing; 1.0 means perfect ranking. AUC is threshold-independent.

Definition — Precision / Recall tradeoff: precision is the fraction of positive predictions that were actually correct; recall is the fraction of actual positives that were correctly caught. Lowering the threshold raises recall and lowers precision; raising it does the reverse.

Real-world example

Real-world example: An email spam filter has to choose a threshold. Set it too low (flag almost anything as spam) and it catches nearly all spam but also buries real emails in the spam folder (high recall, low precision). Set it too high (only flag obvious spam) and legitimate email is safe, but plenty of spam slips through (high precision, low recall). AUC tells you how good the underlying spam-ranking model is regardless of where that cutoff is eventually set; the threshold itself is a business decision made afterward.

Implementation

```
from sklearn.metrics import roc_curve, roc_auc_score, precision_recall_curve

probs = model.predict_proba(X_test)[:, 1]

fpr, tpr, _ = roc_curve(y_test, probs)
auc = roc_auc_score(y_test, probs)

precision, recall, thresholds = precision_recall_curve(y_test, probs)
f1 = 2 * precision * recall / (precision + recall)
best_threshold = thresholds[f1[:-1].argmax()]
```

- `roc_curve` and `roc_auc_score` both take probabilities, not predicted labels — they need the full ranking to sweep every possible threshold themselves
- F1 is the harmonic mean of precision and recall, only scoring high when both are reasonably high at once — a common way to pick one threshold automatically instead of eyeballing a curve

Day 19 — the full pipeline, saved and reloadable

Definition

Definition — Model Persistence: saving a trained model's learned parameters to disk (with `joblib` or `pickle`) so it can be reloaded and used for predictions later without retraining from scratch.

Definition — Out-of-fold prediction (`cross_val_predict`): generates a prediction for every training row using a model that never saw that row during training, by stitching together the held-out predictions from each cross-validation fold. Lets you make decisions (like threshold choice) using training data only, without contaminating the test set.

Real-world example

Real-world example: A company doesn't retrain its fraud-detection model from scratch every time a transaction needs scoring — it trains once, saves the model, and loads that saved file into production to score transactions in real time. If the saved file doesn't also remember the exact decision threshold and input format it was trained with, it's not actually usable by anyone else on the team.

Implementation

```

import joblib
from sklearn.model_selection import cross_val_predict

oof_probs = cross_val_predict(model, X_train, y_train, cv=5, method="predict_proba")[:, 1]
# ... choose best_threshold from oof_probs, same F1 sweep as Day 18 ...

model.fit(X_train, y_train)

bundle = {"model": model, "threshold": best_threshold, "feature_columns": list(X.columns)}
joblib.dump(bundle, "final_model.joblib")

reloaded = joblib.load("final_model.joblib")
reloaded["model"].predict_proba(X_test) # works identically to the original

```

- the threshold and feature column order are saved inside the same dictionary as the model — a model file without them is not a complete, usable deliverable
- always reload the file and compare predictions against the original in the same run before trusting it — a file that was never verified to reload correctly isn't a tested deliverable

Interview prep quiz — technical

Questions about parameters, function behavior, and implementation details — the kind that check whether you've actually written the code, not just read about it.

Q: What does `.predict_proba()` return that `.predict()` doesn't, and how is `.predict()` actually computed from it?

A: `predict_proba()` returns a probability for each class (columns summing to 1 per row); `predict()` is exactly that probability compared against a threshold, 0.5 by default. `predict()` throws away the actual confidence, keeping only the final label.

Q: Why use median rather than mean to fill missing Age values in a dataset like Titanic?

A: Age is right-skewed (a smaller number of much older passengers). The mean would be pulled upward by those outliers; the median is more robust to skew and better represents a 'typical' passenger.

Q: What does `pd.get_dummies(..., drop_first=True)` do, and why drop the first category?

A: It one-hot encodes a categorical column into multiple binary columns, then drops one category's column because it's redundant — its value can always be inferred from the others (if `Embarked_C` and `Embarked_Q` are both 0, the passenger embarked at S).

Q: What's the practical difference between `max_depth` and `min_samples_leaf` in a decision tree, and what does each do to overfitting?

A: `max_depth` caps how many sequential questions the tree can ask; `min_samples_leaf` forces each final leaf to cover a minimum number of rows. Both restrict how finely the tree can carve up the data, which reduces its ability to memorize training-set noise.

Q: What does `cv=5` mean inside `cross_val_score`, and what does the function return?

A: 5-fold cross-validation: the training data is split into 5 parts, the model is trained on 4 and tested on 1, five times with a different held-out fold each time. It returns an array of 5 scores, not a single number.

Q: Why does `GridSearchCV` need its own hyperparameter grid per model type instead of one shared grid?

A: Different model classes expose different constructor arguments — a logistic regression doesn't have a `max_depth`, and a decision tree doesn't have a `C`. Each model's grid has to match its own actual hyperparameter names, or the fit call raises an error.

Q: Why do `roc_curve` and `roc_auc_score` take predicted probabilities as input instead of predicted labels?

A: Both functions need to sweep every possible decision threshold internally to compute the curve or the ranking-based AUC score. A predicted label has already committed to one threshold (0.5), which throws away the information needed to evaluate every other threshold.

Q: What's the purpose of `joblib.dump()` and `joblib.load()`, and what should be saved alongside the model itself?

A: `joblib` serializes a fitted Python object (like a trained scikit-learn model) to disk and reconstructs it later without retraining. Alongside the model, you should save anything needed to use it correctly later —

commonly the chosen decision threshold and the exact list/order of feature columns it expects.

Interview prep quiz — theoretical / conceptual

Questions about why techniques exist and when to reach for them — the kind that check understanding, not memorization.

Q: Why is accuracy alone often a misleading metric, and what would you report instead?

A: Accuracy can look high on an imbalanced dataset just by predicting the majority class every time. Precision, recall, F1, and AUC (or a full `classification_report`) give a clearer picture of how the model performs on each class specifically, especially the minority class that usually matters most.

Q: When would you choose a single decision tree over a random forest, given the forest usually performs better?

A: When interpretability matters more than raw performance — a single tree can be drawn out and explained step by step to a non-technical stakeholder, while a forest of 200 trees cannot. Also useful as a fast baseline before committing to a heavier ensemble.

Q: Why is a single train/test split not sufficient to claim 'my model gets 82% accuracy'?

A: That number reflects performance on one particular random partition of the data. A different split could easily produce a meaningfully different score. Cross-validation gives a distribution of scores (mean and standard deviation) that's a far more honest and reproducible claim.

Q: Why should hyperparameter tuning never be evaluated using the same test set you'll report final performance on?

A: If you repeatedly check test-set performance while adjusting hyperparameters, you're implicitly fitting to the test set's quirks, even without directly training on it. The reported final number stops being an honest estimate of performance on truly unseen data. Tuning decisions should be made with cross-validation on training data only.

Q: In a medical screening context, would you prioritize precision or recall, and why?

A: Usually recall — missing an actual positive case (a real illness) is typically far more costly than a false alarm that gets ruled out by a follow-up test. This is a case where the 'right' threshold is a judgment call based on the cost of each type of error, not just whichever threshold maximizes accuracy.

Q: What does it mean for AUC to be 'threshold-independent', and why is that useful when comparing two models?

A: AUC measures how well a model ranks positives above negatives across every possible threshold at once, rather than being tied to one specific cutoff. It lets you compare two models' underlying quality fairly, before either of you has committed to a business decision about where to set the cutoff.

Q: Why is choosing a decision threshold using out-of-fold predictions (`cross_val_predict`) more defensible than choosing it using the test set?

A: Choosing a threshold based on test-set performance lets test-set information leak into a modeling decision, making the final reported performance optimistic. Out-of-fold predictions are generated entirely from training data, so the threshold decision and the final test-set evaluation stay fully independent.

Q: What's the risk of only saving a trained model file without also saving its decision threshold and expected feature columns?

A: Anyone reloading the model later has no way to know which probability cutoff to apply or what order/format the input features need to be in, making the file effectively unusable or, worse, silently wrong if they guess incorrectly.